

叙事方法论

为什么 V2 做主线，V1 做支撑？

本质问题

面试不是罗列做过什么，而是证明你能从业务域目标态出发，设计一套可运行、可治理、可验证的 AI 原生体系。

V1 讲法：项目执行视角

主线选择

V2 讲法：业务域重构视角



切入点

审核提效 / 知识库 / 工具助手



优势

贴近落地，容易讲清具体工作



局限

容易被听成单点 RAG、Copilot 或提效项目



适用场景

被追问业务痛点、技术实现、指标结果时补充细节



切入点

风控审核业务域的 AI Native Agent 化



优势

有终局感、体系感、操盘感



核心

五条主线 × 三大支柱 × 运行闭环



适用场景

开场主叙事、架构表达、高阶追问、个人贡献表达

用 V2
打开高度，
用 V1
支撑可信度

叙事顺序

1



先讲定位

不是审核助手，
而是业务域
Agent 化

2



再讲矛盾

知识、流程、
工具、权限、
反馈割裂

3



再讲架构

五条主线
×
三大支柱

4



最后讲落地

单次审核闭环、
反馈进化、技术
与指标

★ 亮点保留

- ✓ V2 提供高度：从目标态倒推系统设计
- ✓ V1 提供细节：业务痛点、模块实现、指标证明
- ✓ 二者不是二选一：V2 是主线，V1 是证据

⚠ 误区边界

- ✗ 不要一开场讲工具：容易被压成 RAG 项目 → 先讲业务域问题与目标态
- ✗ 不要只讲愿景：必须落到证据链、Harness、人工复核、评估指标 → 体系 + 证据链 + 治理约束 + 评估验证
- ✗ 不要编造数字：没有真实数据时用指标框架表达 → 指标框架 > 编造数据



面试主线用 V2 建立“为什么重要”，
追问答辩用 V1 证明“我真的做过、能落地”。



项目定位与五层总地图

从审核助手，到 AI 原生风控审核体系

1 本质定位



风控审核业务域的 AI Native Agent 化

不是单点审核助手，也不是风控知识库，而是把知识、流程、工具、权限、证据、反馈重构为 Agent 可运行的领域体系。

一句话定位

以终为始，重构风控审核业务域的知识、流程、工具、权限、证据与反馈，构建可运行、可约束、可解释、可评估、可进化的 AI 原生体系。

价值坐标



更高效
提速降本



更可靠
统一口径



可进化
持续迭代

2 五层叙事

本层关键回答

1



项目定位

业务域重构，不是单点工具

我们要解决什么问题，为什么要做这个项目？

2



核心矛盾

知识、流程、工具、权限、反馈割裂

传统体系为什么难以支撑 AI 稳定接管复杂审核？

3



领域架构

五条主线 × 三大支柱

整个业务域如何被 Agent 化改造？

4



运行闭环

单次审核 × 长期进化

一次审核如何完成，体系如何持续进化？

5



价值证明

短期提效 / 中期提质 / 长期进化

我们如何证明价值，并不断放大价值？

3 亮点



以终为始

先定义目标态，再倒推改造对象，避免在局部最优里打转。



从工具到体系

重构知识、流程、工具、权限、证据、反馈的协同方式，而非新增一个入口。



讲业务域

按风险生命周期组织叙事，覆盖发现、研判、处置、治理与进化全链路。

4 误区边界

不要讲



审核 Copilot

只是给审核员加一个智能助手



单点 RAG

只是一个更智能的知识搜索



模型替代审核员

让模型直接做决策和处置

VS

应该讲



Agent 输出可复核的审核建议，辅助审核员提效



领域知识资产可治理、可组装、可验证、可进化



关键动作受 Harness 与人工复核约束，风险可控可追溯

“

核心不是“AI 帮人审得更快”，
而是让风控审核走向 Agent 驱动、知识约束、工具协同和受控反馈进化。



Agent 驱动



知识约束



工具协同



受控反馈



持续进化



核心矛盾与目标态

传统问题不是审核慢，而是体系割裂

本质问题



AI 很难稳定接管复杂审核，不是因为缺模型，
而是因为知识、流程、工具、权限、反馈分散在不同系统与人脑里。

传统割裂



规则在规则系统里



SOP 在文档里



案例在工单里



经验在审核员脑里



数据在不同工具里



质检申诉在反馈链路外



AI 原生目标态



知识可消费



上下文可组装



工具可调用



权限可约束



证据可解释



反馈可进化

亮点



诊断更深：不是效率问题，而是领域能力组织方式问题



目标更稳：Agent 组织任务、查证证据、形成建议



可控前置：越靠近处置与规则更新，越需要评审、灰度、回滚

误区边界



误区：讲成“减少审核员工作量”

修正：提效只是短期价值，
核心是体系可持续进化



误区：讲成“Agent 自动决策”

修正：Agent 是受控执行者和证据组织者，
不绕过强规则与人工复核

“ 传统体系不是缺一个 AI，而是缺一套让 AI 能读懂、能调用、能受控、
能被验证的领域运行结构。 ”

领域架构母图

五条主线 × 三大支柱



本质问题

整个风控审核业务域如何被 Agent 化改造？

五条主线是业务生命周期，三大支柱是共享能力底座。

五条主线（业务生命周期）



★ 亮点

	操盘视角	主线之间最重要的是反馈信号 它让整个体系持续自我修正和进化
	复用视角	三大支柱避免重复造轮子 沉淀为平台能力，支撑多业务域复用
	演进视角	修正信号反哺知识、工具、Harness、评估集 让体系从经验驱动走向数据与证据驱动

⚠ 误区边界

不要讲成 五个孤立模块	→	它是一条 风险事件生命周期 有输入→处理→输出→反馈→进化
不要把支柱等同 RAG、Tool、Workflow	→	支柱是领域能力抽象， 技术是实现方式 实现可以更换，能力抽象要稳定



五条主线回答业务域怎么拆，三大支柱回答能力怎么复用，运行闭环回答系统怎么持续进化。

运行闭环

单次审核 × 长期进化

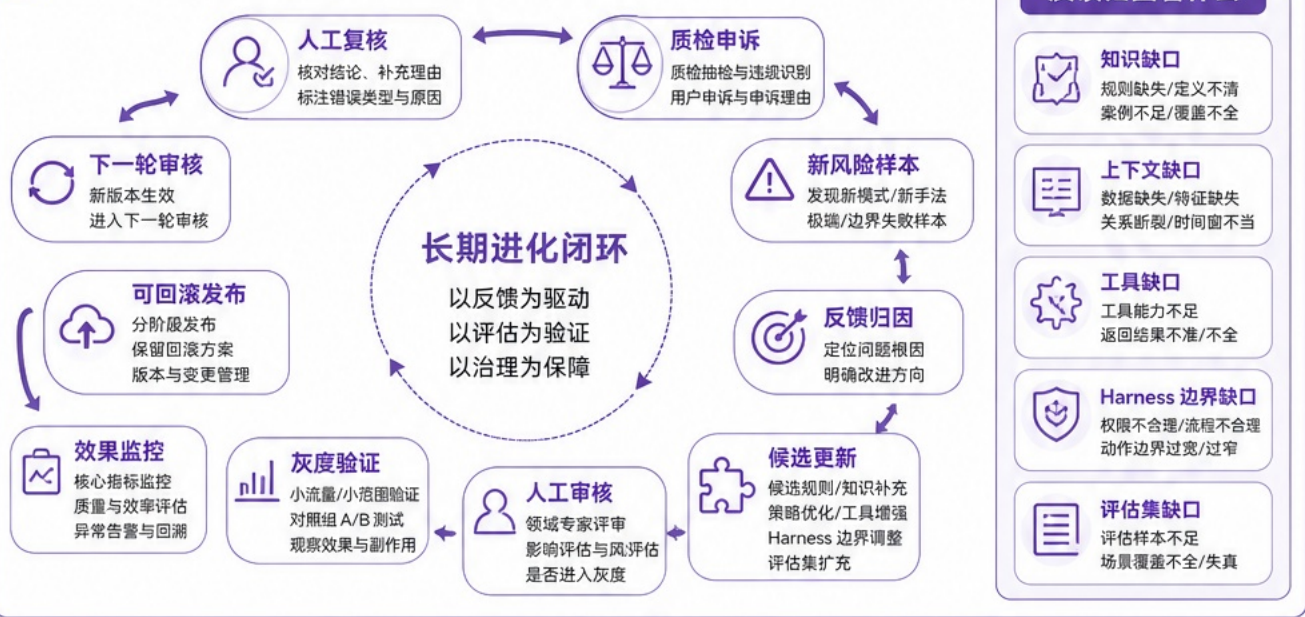
本质问题

Agent 如何完成一次审核？每次审核产生的修正信号如何受控进入体系更新？

1 单次审核闭环：交付可解释、可复核的审核建议



2 长期进化闭环：交付可验证、可回滚的体系更新



★ 亮点

- 证据驱动：**事实来自工具、日志、规则、案例和知识资产
- 边界清晰：**高风险、低置信、规则冲突、高影响处置必须转人工
- 反馈受控：**反馈不直接写回正式体系，必须评审、灰度、回滚

⚠ 误区边界

不要这么说

- ✗ Agent 自动处罚用户
- ✗ Agent 自动改规则
- ✗ 模型直接替代审核员

应该这么说

- ✓ Agent 输出证据链与建议；关键动作受 Harness 与人工复核约束



一次审核交付可解释建议；长期闭环交付可验证、可回滚的体系进化。

技术支撑与价值证明

技术不是主角，技术服务主叙事

本质问题

技术要证明这套 AI 原生审核体系为什么能稳定运行、能被验证、能持续进化。

技术模块



支撑能力



面试表达



RAG / 知识库

审核域知识治理

- 结构化管理规则、SOP、案例、质检标准、判例
- 分层标签、权限隔离、版本管理、更新可追溯



规则、SOP、案例、质检标准可被 Agent 消费



Tool Calling / 数据服务

案件上下文与证据工具

- 多源数据接入：日志、交易、设备、关系、名单等
- 工具调用：检索、统计、关联分析、相似案例召回



查日志、看关系、召回案例、形成证据



Workflow / 编排

案件流转与人机协同

- 流程编排：分流、并行、会签、升级、回退、结束
- 人机协同：任务分配、节点留痕、状态可视化



低中高风险进入不同处理路径



Harness / 权限边界

风险门控与动作约束

- 权限边界：数据、工具、动作
- 风险门控：风险类型、置信度、规则冲突、高影响处置
- 强制策略：人工复核、二次确认、审计留痕、可回滚



高风险、低置信、高影响动作必须受控



Eval / 回归测试

质量评估与边界覆盖

- 评估集：典型案例、边界案例、新风险样本
- 评估维度：正确性、覆盖度、规则引用、证据完整性



误判、漏判、规则冲突、新风险样本可评估



Observability / 可观测性

运行追踪与反馈归因

- 全链路追踪：请求、工具调用、证据来源、模型输出、人工处理
- 反馈归因：误判原因、漏判原因、规则缺失、数据缺失



工具调用、证据来源、效果监控可追踪

价值三层

短期提效



- 审核耗时下降
- 查证步骤减少
- 证据链生成提速
- 中低风险分流



中期提质



- 审核口径统一
- 证据完整率提升
- 规则引用准确
- 复核质量提升



长期进化



- 新风险样本回流
- 知识持续更新
- Harness 边界优化
- 评估集扩充

指标框架



效率指标

- 单案件耗时
- 查证步骤
- 证据生成耗时



质量指标

- 采纳率
- 证据完整率
- 误判漏判
- 申诉修正



体系指标

- 反馈归因完成率
- 知识覆盖
- 评估集新增
- 回滚记录

★ 亮点

- 1 先讲业务问题，再讲领域架构，最后讲技术如何证明体系成立
- 2 不把技术清单当主叙事
- 3 同时看效率、质量、体系三类指标

⚠ 误区边界

- ❌ 不要只讲用了 RAG、Tool、Workflow → 要讲支撑哪条主线
- ❌ 没有真实数据不要编百分比 → 用指标框架表达
- ❌ 不只看模型效果 → 看业务闭环与体系进化



技术要放回业务闭环里：它支撑什么主线、控制什么风险、证明什么价值。